

TP 1 : Space Invaders & Sous-Système Audio

Master Programmation Jeu Vidéo — Architecture C++ & Raylib

Master Programmation Jeu Vidéo

24 septembre 2026

! Informations & Modalités Clés

- **Objectif** : Réimplémenter intégralement *Space Invaders* et lui adjoindre un sous-système audio dynamique.
- **Organisation** : **Travail strictement individuel** (les binômes et le partage de code sont formellement interdits).
- **Poids dans la note** : Ce TP compte pour **10% de la note finale** du module.
- **Date limite de rendu** : **Jeudi 24 septembre 2026 à 23h59**.
- **Stack imposée** : C++ (standard C++17 ou C++20), CMake (avec `FetchContent` pour Raylib 5.0) et Raylib.

1. Contexte & Objectifs Pédagogiques

Dans la continuité directe du Cours 2, l'objectif de ce TP est de concevoir un jeu 2D complet en C++ moderne selon les principes du **Data-Oriented Design** :

- **Discipline mémoire bas niveau** : Bannir l'allocation dynamique sur le tas dans la boucle de simulation ; privilégier la pile (*Stack*) et les tableaux contigus.
- **Déterminisme temporel** : Mouvements strictement indépendants du framerate grâce au Delta Time (Δt).
- **Architecture logicielle découplée** : Séparation nette entre capture des entrées, logique de simulation (*Update*) et rendu (*Draw*).
- **Intégration multimédia** : Exploiter le module audio de Raylib pour sonoriser le jeu sans compromettre la fluidité temps réel.

2. Contraintes Techniques Non Négociables

Toute entorse à ces règles architecturales fondamentales sera sanctionnée :

- **Zéro allocation dynamique dans la boucle de jeu** : Aucun appel à `new`, `delete`, `malloc()`, `free()`, ni redimensionnement dynamique (`std::vector::push_back`) n'est toléré pendant l'exécution de la boucle principale (`while (!WindowShouldClose())`). Toutes les entités doivent résider dans des pools statiques à taille fixe.
- **Discipline de passage des arguments** :
 - **Passage par référence modifiable** (`Entity&`) lorsqu'une fonction altère l'état d'un objet (interdiction des copies par valeur).
 - **Passage par référence constante** (`const Entity&`) pour les fonctions de consultation et de rendu pur (`DrawGame`).
- **Autonomie de compilation CMake** : Le fichier `CMakeLists.txt` doit intégrer Raylib 5.0 de manière autonome via `FetchContent`. Le projet doit se configurer et se compiler sans dépendance externe préinstallée via : `cmake -B build` puis `cmake --build build`.

3. Spécifications Fonctionnelles : Le Canon & Les Projectiles

3.1 Canon du Joueur & Déplacements (Jalon 1)

- Modélisation via une structure plate `Player` (position, dimensions, vitesse de déplacement).
- Déplacement horizontal fluide aux touches fléchées (`KEY_LEFT`, `KEY_RIGHT`) ou A / D.
- Mouvement pondéré par le Delta Time (`GetFrameTime()`) : $P_x = P_x \pm V \cdot \Delta t$.
- Clamping rigoureux du canon dans les limites de l'écran : `[0, SCREEN_WIDTH - size.x]`.
- Extraction de la logique dans une fonction dédiée : `void UpdatePlayer(Player& player, Bullet bullets[], float dt)`.

3.2 Pool de Projectiles & Cadence de Tir (Jalon 2)

- Modélisation d'une structure `Bullet` avec drapeau booléen d'activité (`active`).
- Tableau contigu fixe de balles alloué sur la pile (`MAX_BULLETS = 10`).
- Déclenchement du tir à la barre d'espace (`KEY_SPACE`) avec recherche linéaire du premier emplacement libre.
- Temporisateur de cadence de tir (`fireCooldown`) empêchant le tir continu en rafale lors du maintien de la touche.
- Recyclage immédiat des tirs sortant par le haut de l'écran (`active = false`, aucune libération mémoire).

4. Spécifications Fonctionnelles : Flotte d’Envahisseurs & Collisions

4.1 Grille d’Aliens & Localité Spatiale (Jalon 3)

- Grille de 55 envahisseurs (5 lignes × 11 colonnes) stockée à plat dans un tableau 1D contigu.
- Formule d’indexation 2D vers 1D : $\text{index} = \text{row} \times \text{ALIEN_COLS} + \text{col}$.
- Différenciation visuelle des rangées d’aliens par codes couleurs (ex: Rouge, Orange, Bleu ciel).
- Affichage tête haute (HUD) du score et du nombre d’aliens vivants restants (`DrawText`).

4.2 Machine à États de la Flotte & Collisions AABB (Jalon 4)

- **Marche collective cadencée** : Déplacement horizontal de la flotte par impulsions discrètes pilotées par `fleetStepTimer`.
- **Rebond sur bordure & Descente** : Dès qu’un alien vivant atteint le bord gauche ou droit, le sens horizontal s’inverse et toute la formation descend d’un cran vertical (+18 px).
- **Accélération dynamique** : L’intervalle de déplacement de la flotte doit diminuer à mesure que les aliens sont détruits :

$$\text{interval} = 0.05 + 0.45 \times \left(\frac{\text{aliens_vivants}}{\text{total_aliens}} \right)$$

- **Collisions AABB** : Test boîte-à-boîte (`CheckCollisionRecs`) entre chaque balle active et chaque alien vivant. En cas d’impact : désactivation instantanée de la balle et de l’alien, +10 points au score, et mise à jour du compteur.
- **Fin de partie** : Victoire lorsque la flotte est anéantie ; Défaite si un alien vivant atteint la ligne de défense du joueur.

Extension Obligatoire : Intégration du Sous-Système Audio

Le jeu doit obligatoirement être sonorisé en exploitant l’API audio native de Raylib :

- **Initialisation & Fermeture** : Initialiser le périphérique avec `InitAudioDevice()` au lancement et libérer les ressources audio avec `CloseAudioDevice()` à la sortie (avant `CloseWindow()`).
- **Gestion stricte des ressources sonores** :
 - Placer les fichiers audio (.wav ou .ogg) dans un sous-dossier `assets/` à la racine du projet.
 - Charger les sons une seule fois à l’initialisation via `LoadSound()`, et les libérer à la sortie avec `UnloadSound()`.
 - **Interdiction formelle d’appeler `LoadSound()` à l’intérieur de la boucle de jeu.**
- **Événements sonores minimaux requis (au moins 3 sons distincts)** :
 1. *Tir laser* : Joué à chaque nouveau projectile émis par le joueur (`PlaySound()`).
 2. *Destruction d’alien* : Déclenché lors d’un impact réussi balle-alien.
 3. *Battement de marche* : Bruit cadencé joué à chaque pas d’impulsion de la flotte.
 4. (*Optionnel / Bonus*) : Jingle de victoire ou son de Game Over.

Critères d’Évaluation (Poids : 10% de la Note Finale)

Ce travail pratique est évalué dans sa globalité et compte pour **10% de la note finale du module** :

Axe d’Évaluation	Attendus & Exigences Clés
Discipline Zéro-Heap & Mémoire	Aucune allocation dynamique sur le tas dans la boucle de jeu (<code>new</code> , <code>push_back</code>). Utilisation exclusive de tableaux contigus à taille fixe. Respect rigoureux des passages par référence (&) et référence constante (<code>const &</code>).
Mécaniques & Boucle de Jeu	Clamping fluide, indépendance au framerate via Δt , cadence de tir opérationnelle, collisions AABB fiables, machine à états et accélération de la flotte, gestion propre des conditions de victoire et de défaite.
Sous-Système Audio Raylib	Cycle de vie rigoureux (<code>InitAudioDevice</code> , <code>LoadSound</code> , <code>PlaySound</code> , <code>UnloadSound</code> , <code>CloseAudioDevice</code>), sons synchronisés aux événements de jeu, zéro fuite mémoire et zéro allocation audio en cours de partie.
Qualité du Code & Build CMake	Portabilité totale du <code>CMakeLists.txt</code> (<code>FetchContent</code>), clarté du découpage procédural (<code>Init</code> , <code>Update</code> , <code>Draw</code>), conventions de nommage explicites, propreté et lisibilité du C++.

Modalités de Rendu & Consignes

- **Date limite impérative** : Jeudi 24 septembre 2026 à 23h59.
- **Rendu** : en classe